MINI PROJECT REPORT

on

PET CLINIC MANAGEMENT SYSTEM

A Database Management System using MySQL

Subject: Database Management System (DBMS)

Submitted by	
Name	Khushi Phogat
Course	BCA Data Science
Subject	Database Management System (DBMS)
Academic Session	2026–2027

Submitted to	Mrs. Harkamal Kaur

GitHub Repository

https://github.com/justkhushii/pet-clinic-management-system

Table of Contents

1. Introduction..... 3
2. Objectives..... 3
3. Problem Statement..... 3
4. Modules of the Project..... 4
5. Tools and Technologies Used..... 4
6. Database Design..... 5
7. SQL Code..... 7
8. Code with Outputs..... 9
9. Relationships Between the Tables..... 12
10. Testing and Result Analysis..... 13
11. Advantages of the System..... 13
12. Conclusion..... 14
13. Future Scope..... 14
GitHub Repository Link..... 14

1. Introduction

The Pet Clinic Management System is a database project designed to store and manage information about pet owners, their pets, clinic appointments, and treatments. A clinic needs accurate records to keep track of pets, understand who owns each pet, schedule visits, and record treatment costs. Using MySQL, this project stores the information in related tables so that records can be searched and viewed efficiently.

The project uses four tables: Owners, Pets, Appointments, and Treatments. Primary keys identify individual records, while foreign keys connect pets to owners and appointments and treatments to pets.

MySQL is a widely used open-source relational database management system (RDBMS). Data is stored in tables made of rows (records) and columns (attributes), and the tables can be linked with each other. MySQL Workbench was used as the graphical tool to write the SQL statements and to view the results of the queries.

The system created in this project is a small but complete example of a database that supports the daily work of a pet clinic. It covers four main areas: who the owners are, which pets they own, when each pet is scheduled to visit, and what treatment was given and at what cost.

1.1 Scope of the Project

- The project covers database creation, table design, data insertion and data retrieval using SQL.
- It demonstrates one-to-many relationships between owners and pets, pets and appointments, and pets and treatments.
- It does not include a front-end application; the database is accessed directly through MySQL Workbench.

1.2 Need for a Database System

Many small clinics still keep records in registers or loose files. This leads to repeated data, lost records, slow searching and mistakes in billing. A database system solves these problems by keeping every fact in one place, enforcing rules on the data, and allowing quick queries. The Pet Clinic database applies these ideas on a small scale.

2. Objectives

- To maintain basic information about pet owners and their contact details.
- To store pet details such as name, species, breed, age, and owner.
- To record appointments, dates, reasons, and appointment status.
- To maintain treatment information, dates, and costs.
- To connect related records using primary keys and foreign keys.
- To retrieve useful information using SQL SELECT and JOIN queries.
- To reduce data duplication and keep the data consistent through relational design.
- To understand how the DDL (CREATE) and DML (INSERT, SELECT) commands of SQL work together in a real mini project.

3. Problem Statement

When pet clinic records are maintained manually, it may be difficult to search for an owner's details, identify the owner of a pet, track appointments, or review treatment costs. This project provides a simple relational database that stores these details in an organized way and allows users to retrieve related information through SQL queries.

The main problems of a manual system that this project addresses are listed below.

- **Slow searching:** Searching for one owner or pet in paper records takes a lot of time.
- **Duplicate data:** The same owner or pet details may be written many times in different places.
- **Poor linking of records:** It is difficult to know which pet belongs to which owner, or which appointment belongs to which pet.
- **Billing errors:** Treatment costs written by hand can be wrong and are hard to add up.
- **Risk of loss:** Paper records can be damaged or lost, while a database can be backed up.

4. Modules of the Project

- **Owners Module:** Stores each owner's ID, name, phone number, and address.
- **Pets Module:** Stores pet name, species, breed, age, and the ID of its owner.
- **Appointments Module:** Stores appointment date, reason, and status for a pet.
- **Treatments Module:** Stores treatment name, cost, and treatment date for a pet.
- **Reports and Search Module:** Uses SELECT and JOIN queries to display records and related details.

All five modules work together. The Owners module feeds the Pets module, the Pets module feeds the Appointments and Treatments modules, and the Reports module reads from all of them.

5. Tools and Technologies Used

- Database: MySQL
- SQL Editor / GUI: MySQL Workbench
- Language: SQL (Structured Query Language)
- Operating System: Microsoft Windows
- Documentation: Microsoft Word

The table below summarises how each tool was used in this project.

Tool	Purpose
MySQL Server	Stores the PetClinic database and executes the SQL statements.
MySQL Workbench	Graphical client used to write queries, run them and capture result screenshots.
SQL	Language used for creating tables (DDL), inserting data (DML) and retrieving data (queries).
Microsoft Word	Used to prepare and print this project report.

6. Database Design

The database is named PetClinic and contains the following four tables:

- Owners (owner_id, owner_name, phone, address)
- Pets (pet_id, pet_name, species, breed, age, owner_id)
- Appointments (appointment_id, pet_id, appointment_date, reason, status)
- Treatments (treatment_id, pet_id, treatment_name, cost, treatment_date)

Relationships: One owner can have multiple pets. Each pet can have multiple appointments and treatment records. The owner_id field in Pets references Owners(owner_id). The pet_id fields in Appointments and Treatments reference Pets(pet_id).

6.1 Entity Relationship Diagram

The ER diagram below shows the four tables, their primary keys (PK), their foreign keys (FK) and the one-to-many (1 : N) relationships between them.

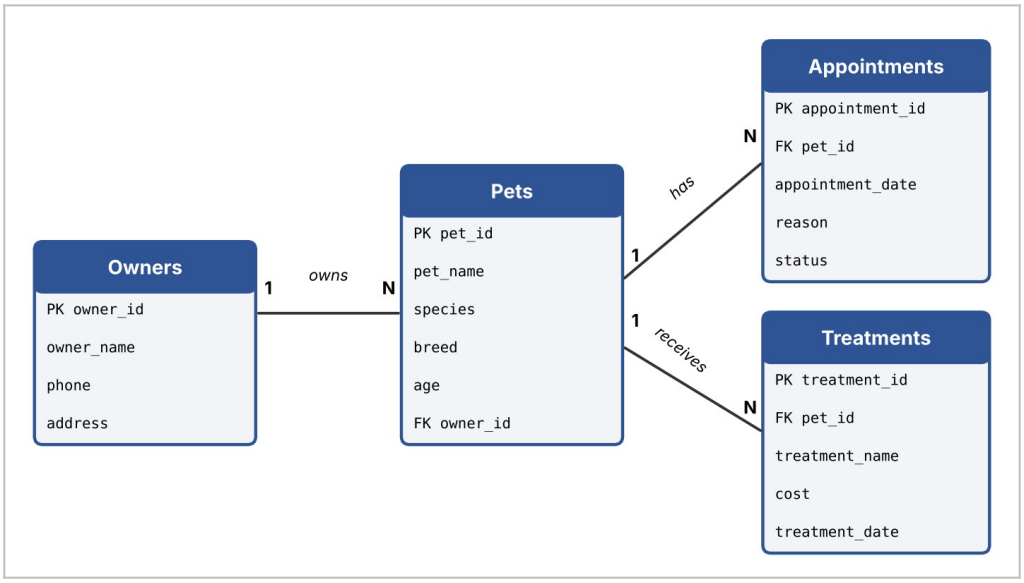


Diagram 1: ER diagram of the PetClinic database

6.2 Data Dictionary

The data dictionary lists every column of every table, with its data type and constraint.

Table 1: Owners

Column	Data type	Constraint	Description
owner_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the owner
owner_name	VARCHAR(100)	NOT NULL	Full name of the owner
phone	VARCHAR(15)	-	Contact number
address	VARCHAR(200)	-	City / address of the owner

Table 2: Pets

Column	Data type	Constraint	Description
pet_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the pet
pet_name	VARCHAR(100)	NOT NULL	Name of the pet
species	VARCHAR(50)	-	Dog, Cat, etc.
breed	VARCHAR(50)	-	Breed of the pet
age	INT	-	Age in years
owner_id	INT	FOREIGN KEY -> Owners(owner_id)	Owner of the pet

Table 3: Appointments

Column	Data type	Constraint	Description
appointment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the appointment
pet_id	INT	NOT NULL, FOREIGN KEY -> Pets(pet_id)	Pet that has the appointment
appointment_date	DATE	-	Date of the visit
reason	VARCHAR(200)	-	Reason for the visit
status	VARCHAR(50)	-	Scheduled, Completed, etc.

Table 4: Treatments

Column	Data type	Constraint	Description
treatment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the treatment
pet_id	INT	NOT NULL, FOREIGN KEY -> Pets(pet_id)	Pet that received the treatment
treatment_name	VARCHAR(100)	-	Name of the treatment
cost	DECIMAL(8,2)	-	Cost of the treatment
treatment_date	DATE	-	Date of the treatment

6.3 Keys and Constraints Used

- **PRIMARY KEY:** A primary key uniquely identifies every row in a table. Here owner_id, pet_id, appointment_id and treatment_id are primary keys.
- **FOREIGN KEY:** A foreign key links a column to the primary key of another table, which keeps the data consistent. A pet cannot be added for an owner that does not exist.
- **NOT NULL:** Columns that must always contain a value, such as owner_name, pet_name and the pet_id link in Appointments and Treatments, are declared NOT NULL.

- **AUTO_INCREMENT:** The ID columns generate the next number automatically, so the user does not have to type it.

6.4 Normalization

The design follows the first three normal forms. Every column holds a single value (1NF). Every non-key column depends on the whole primary key (2NF), and no non-key column depends on another non-key column (3NF). For example, owner details are stored only once in the Owners table instead of being repeated in every pet record, so an address change needs to be made in one place only.

7. SQL Code

This section contains the complete SQL script of the project. Statements are written in the order in which they must be executed: first the database, then the tables (parent tables before child tables) and finally the data.

7.1 Create the database

```
CREATE DATABASE PetClinic;
USE PetClinic;
```

7.2 Create the Owners table

```
CREATE TABLE Owners (
    owner_id INT PRIMARY KEY AUTO_INCREMENT,
    owner_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    address VARCHAR(200)
);
```

7.3 Create the Pets table

```
CREATE TABLE Pets (
    pet_id INT PRIMARY KEY AUTO_INCREMENT,
    pet_name VARCHAR(100) NOT NULL,
    species VARCHAR(50),
    breed VARCHAR(50),
    age INT,
    owner_id INT,
    FOREIGN KEY (owner_id) REFERENCES Owners(owner_id)
);
```

7.4 Create the Appointments table

```
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,
    pet_id INT NOT NULL,
    appointment_date DATE,
    reason VARCHAR(200),
    status VARCHAR(50),
    FOREIGN KEY (pet_id) REFERENCES Pets(pet_id)
);
```

7.5 Create the Treatments table

```
CREATE TABLE Treatments (
    treatment_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    pet_id INT NOT NULL,
    treatment_name VARCHAR(100),
    cost DECIMAL(8,2),
    treatment_date DATE,
    FOREIGN KEY (pet_id) REFERENCES Pets(pet_id)
);
```

7.6 Insert owner records

```
INSERT INTO Owners (owner_name, phone, address)
VALUES
('Khushi Phogat', '9876543210', 'Delhi'),
('Parnika Lodha', '9876501234', 'Chandigarh'),
('Sameer Rajora', '9876512345', 'Bangalore');
```

Note: The phone numbers above are sample values used for this project. Replace them if required.

7.7 Insert pet records

```
INSERT INTO Pets (pet_name, species, breed, age, owner_id)
VALUES
('Bruno', 'Dog', 'Labrador', 3, 1),
('Milo', 'Cat', 'Persian', 2, 2),
('Coco', 'Dog', 'Pug', 4, 3);
```

7.8 Insert appointment records

```
INSERT INTO Appointments (pet_id, appointment_date, reason, status)
VALUES
(1, '2026-10-24', 'Regular checkup', 'Scheduled'),
(2, '2026-10-25', 'Vaccination', 'Scheduled'),
(3, '2026-10-26', 'Dental checkup', 'Scheduled');
```

7.9 Insert treatment records

```
INSERT INTO Treatments (pet_id, treatment_name, cost, treatment_date)
VALUES
(1, 'General Checkup', 500.00, '2026-10-24'),
(2, 'Vaccination', 800.00, '2026-10-25'),
(3, 'Dental Cleaning', 1200.00, '2026-10-26');
```

7.10 Explanation of the SQL Commands

- **CREATE DATABASE / USE:** Creates a new database named PetClinic. USE selects it, so every following statement works inside this database.
- **CREATE TABLE:** Creates a table with its columns, data types and constraints. Owners is created first because Pets refers to it.
- **INSERT INTO ... VALUES:** Adds new rows to a table. The ID columns are left out because AUTO_INCREMENT fills them.
- **SELECT:** Reads data from a table. SELECT * returns all columns.
- **JOIN ... ON:** Combines rows of two tables using a matching column, for example Pets.owner_id = Owners.owner_id.

8. Code with Outputs

The following SQL queries were used to display the records and demonstrate the relationships between tables. The screenshot of each query, as captured from MySQL Workbench, is shown below together with a short explanation of the result.

Output 1: Owners table

```
SELECT * FROM Owners;
```

Purpose: Shows all owner records, including names and cities.

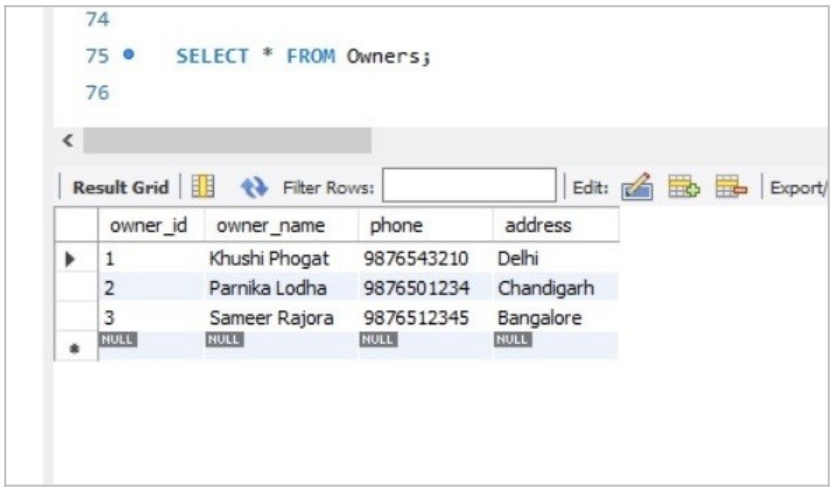


Figure 1: Output of the query for the owners table (MySQL Workbench)

Explanation: The Owners table holds three records. Each owner has a unique owner_id (the primary key) together with a name, a phone number and a city. These owner_id values are later used by the Pets table.

Output 2: Pets table

```
SELECT * FROM Pets;
```

Purpose: Shows the pet details and owner IDs.

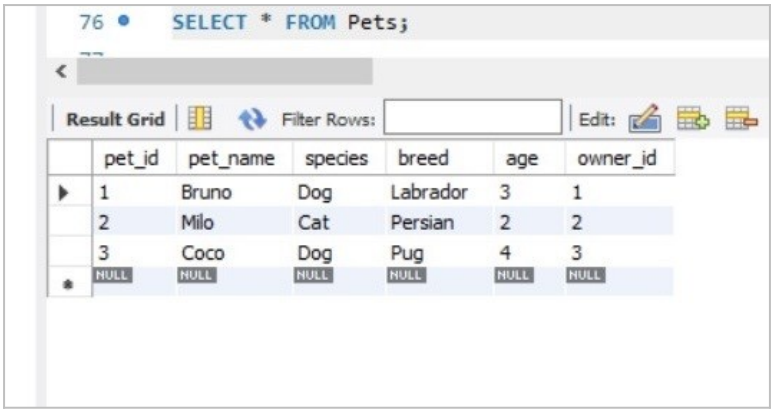


Figure 2: Output of the query for the pets table (MySQL Workbench)

Explanation: The Pets table holds three pets (Bruno, Milo and Coco). The owner_id column is a foreign key, so Bruno belongs to owner 1, Milo to owner 2 and Coco to owner 3.

Output 3: Appointments table

```
SELECT * FROM Appointments;
```

Purpose: Shows appointment dates, reasons, and statuses.

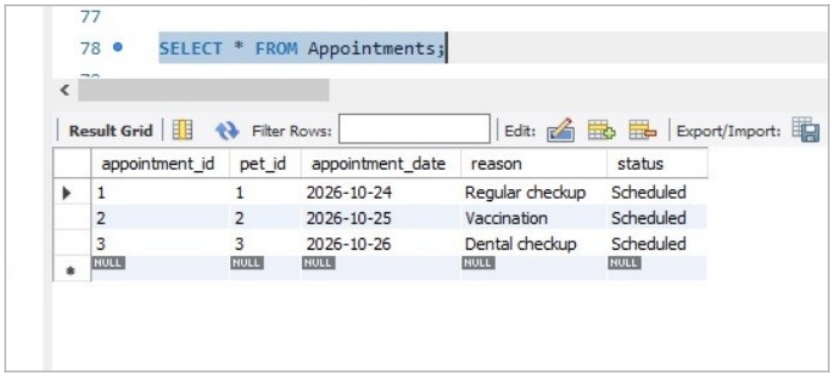


Figure 3: Output of the query for the appointments table (MySQL Workbench)

Explanation: All three appointments are scheduled for 24, 25 and 26 October 2026. Each row points to a pet through pet_id and records the reason for the visit and its status.

Output 4: Treatments table

```
SELECT * FROM Treatments;
```

Purpose: Shows treatment names, costs, and dates.

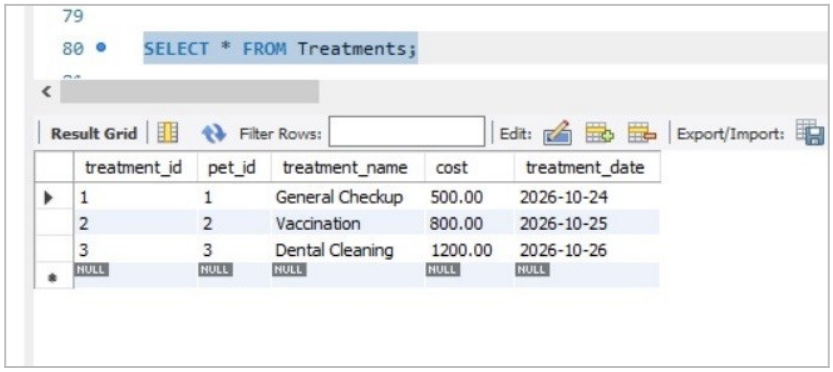


Figure 4: Output of the query for the treatments table (MySQL Workbench)

Explanation: The Treatments table stores the treatment given to each pet along with its cost and date. The costs are 500.00, 800.00 and 1200.00, stored as DECIMAL(8,2).

Output 5: Pets with owner details

```
SELECT Pets.pet_name, Pets.species,
Owners.owner_name, Owners.address
FROM Pets
JOIN Owners ON Pets.owner_id = Owners.owner_id;
```

Purpose: Shows each pet with its owner's name and city.

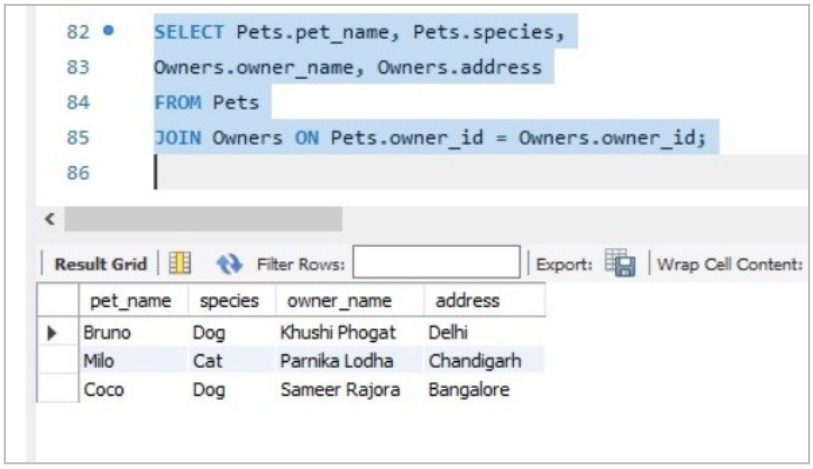


Figure 5: Output of the query for the pets with owner details (MySQL Workbench)

Explanation: The JOIN matches Pets.owner_id with Owners.owner_id, so each pet is shown next to the name and city of its owner: Bruno with Khushi Phogat (Delhi), Milo with Parnika Lodha (Chandigarh) and Coco with Sameer Rajora (Bangalore).

Output 6: Appointment details with pet names

```
SELECT Appointments.appointment_id,
Pets.pet_name, Appointments.appointment_date,
Appointments.reason, Appointments.status
FROM Appointments
JOIN Pets ON Appointments.pet_id = Pets.pet_id;
```

Purpose: Shows appointments together with the related pet names.

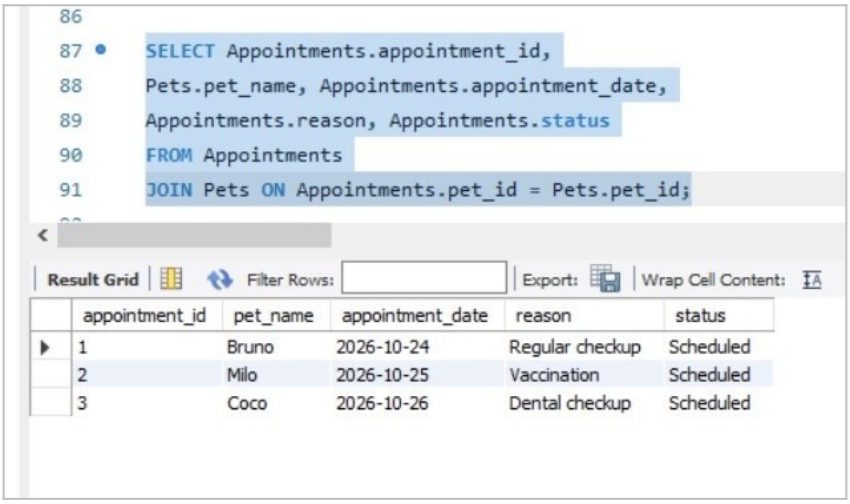


Figure 6: Output of the query for the appointment details with pet names (MySQL Workbench)

Explanation: Appointments does not store pet names, so the JOIN on pet_id brings the name in from Pets. The result is a readable appointment list showing who is coming on which date and why.

Output 7: Treatment details with pets names

```
SELECT Treatments.treatment_id,
Pets.pet_name, Treatments.treatment_name,
Treatments.cost, Treatments.treatment_date
FROM Treatments
JOIN Pets ON Treatments.pet_id = Pets.pet_id;
```

Purpose: Shows treatments and costs together with the related pet names.

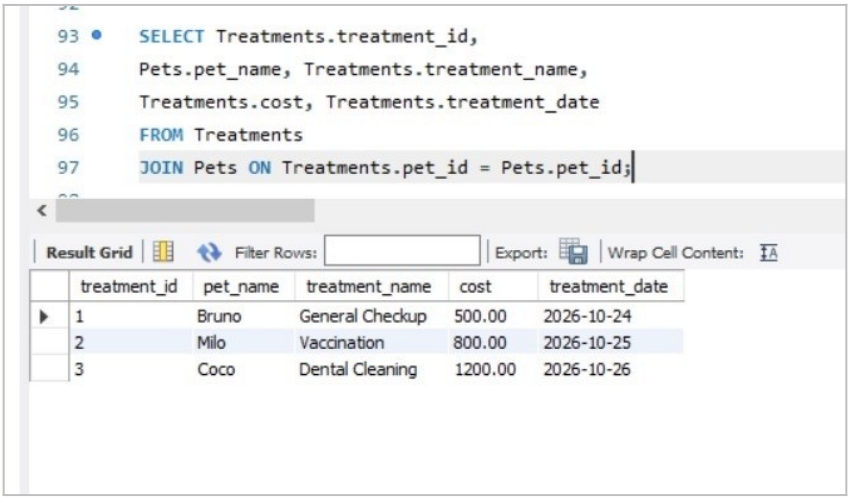


Figure 7: Output of the query for the treatment details with pet names (MySQL Workbench)

Explanation: In the same way, the JOIN on pet_id replaces the numeric pet_id in Treatments with the pet's name, making the treatment and cost records easy to read.

9. Relationships Between the Tables

The most important idea of a relational database is that tables are connected. The three JOIN queries in Section 8 show these connections in practice.

Parent table	Child table	Linking column	Relationship
Owners	Pets	owner_id	One owner can have many pets
Pets	Appointments	pet_id	One pet can have many appointments
Pets	Treatments	pet_id	One pet can have many treatments

9.1 How a JOIN Works

When two tables are joined, MySQL takes each row of the first table and looks for rows in the second table where the linking columns are equal. For example, Bruno has owner_id = 1, and the Owners row with owner_id = 1 is Khushi Phogat. The JOIN therefore returns Bruno together with Khushi Phogat and Delhi.

9.2 Referential Integrity

Because of the foreign keys, MySQL refuses to insert a pet whose owner_id does not exist in Owners, and it refuses to delete an owner who still has pets. This property is called referential integrity and it prevents orphan records in the database.

9.3 Summary of Stored Data

Pet	Species / Breed	Owner	Appointment	Treatment (Cost)
Bruno	Dog / Labrador	Khushi Phogat	24-10-2026 Regular checkup	General Checkup (500.00)
Milo	Cat / Persian	Parnika Lodha	25-10-2026 Vaccination	Vaccination (800.00)
Coco	Dog / Pug	Sameer Rajora	26-10-2026 Dental checkup	Dental Cleaning (1200.00)

The total of the three treatment costs shown above is 500.00 + 800.00 + 1200.00 = 2500.00.

10. Testing and Result Analysis

After the tables were created and the records were inserted, the following checks were performed in MySQL Workbench.

No.	Test	Expected result	Observed result
1	SELECT * FROM Owners;	3 owner records	3 records displayed (Figure 1)
2	SELECT * FROM Pets;	3 pet records with owner IDs	3 records displayed (Figure 2)
3	SELECT * FROM Appointments;	3 scheduled appointments	3 records displayed (Figure 3)
4	SELECT * FROM Treatments;	3 treatment records with costs	3 records displayed (Figure 4)
5	JOIN of Pets and Owners	Each pet with its owner and city	Correct owner shown for every pet (Figure 5)
6	JOIN of Appointments and Pets	Each appointment with the pet name	Correct pet name shown (Figure 6)
7	JOIN of Treatments and Pets	Each treatment with the pet name	Correct pet name shown (Figure 7)

10.1 Result Analysis

All seven queries returned the expected rows. The four SELECT * queries confirm that the data was stored correctly, and the three JOIN queries confirm that the primary key and foreign key relations work as designed. In every JOIN result the number of rows is three, which means no record was lost or duplicated by the join.

11. Advantages of the System

- All clinic data is kept in one organised place.
- Related records are linked, so an owner, a pet, its appointment and its treatment can be seen together.
- Data duplication is avoided through normalization.
- Constraints and foreign keys keep the data correct and consistent.

- Searching and reporting are fast because SQL queries can filter and join data in seconds.
- The design is simple and can be extended with new tables without changing the existing ones.

11.1 Limitations

- There is no graphical front end; staff must use SQL or MySQL Workbench.
- There is no user login or role-based security in the current version.
- Only a small amount of sample data was used.
- Billing, medicines and veterinarian details are not yet stored.

11.2 Learning Outcomes

By completing this project the following skills were practised: designing a relational schema, choosing data types, applying primary and foreign keys, writing CREATE TABLE and INSERT statements, and writing SELECT queries with JOIN to combine data from more than one table.

12. Conclusion

The Pet Clinic Management System demonstrates how a relational database can be used to manage pet owners, pets, appointments, and treatments. MySQL tables, primary keys, foreign keys, INSERT statements, SELECT statements, and JOIN queries were used to store and retrieve related information. The project helped demonstrate the basic concepts of database design and SQL.

The work also showed that a well designed schema makes later queries simple. Because the relationships were defined correctly at the start, the JOIN queries needed only one matching condition each, and the results were accurate. The project can be extended in future into a complete clinic management application.

13. Future Scope

- Add a Veterinarians table to store doctor details.
- Add medicines and prescription records.
- Create a user interface for clinic staff.
- Add search by owner name, pet name, or appointment date.
- Add payment and billing records.
- Add reports such as total income per day or number of appointments per month using GROUP BY and aggregate functions.
- Add user login and access control for the clinic staff.
- Take regular backups of the database.

GitHub Repository Link

GitHub Repository Link:

https://github.com/justkhushii/pet-clinic-management-system

The repository contains the SQL source code, project documentation, and output screenshots of the Pet Clinic Management System.